

Proyecto Integrador de Sistemas de Recuperación de Información. Documentación

Kelen Alfaro García , Adriana Boué García, Carlos Alejandro Mazorra Matos

1 Definición del Dominio Seleccionado

El dominio seleccionado es cultura y entretenimiento, con énfasis en películas y series. El sistema recupera documentos que describen obras audiovisuales (título, sinopsis, género, reparto, dirección, país y año), extraídos desde fuentes web especializadas mediante el crawler/scraper del proyecto

1.1 Objetivo de Recuperación

- Permitir búsquedas semánticas en lenguaje natural sobre películas y series.
- Recuperar resultados relevantes, aunque la consulta no use exactamente las mismas palabras del documento.

1.2 Alcance del Corte 1

- Construcción de un corpus inicial del dominio.
- Generación de representaciones vectoriales densas (embeddings).
- Ranking básico por similitud coseno.

1.3 Alcance del Corte 2

- Refinamiento y depuración del corpus documental.
- Implementación de un modelo de Generación Aumentada por Recuperación (RAG) para ofrecer respuestas sintetizadas en lenguaje natural.
- Incorporación de un módulo de expansión web (web_search) para enriquecer las respuestas con información externa actualizada cuando la consulta lo amerita.
- Ajuste de la infraestructura de almacenamiento mutando a un formato JSON estandarizado para los metadatos y un índice vectorial renovado.

1.4 Alcance del Corte 3

- Incorporación de una interfaz visual en Streamlit para facilitar la exploración del corpus mediante búsquedas en lenguaje natural.

- Incorporación del módulo de posicionamiento para ordenar y priorizar los resultados según señales de relevancia, metadatos y criterios visuales.
- Integración de autenticación de usuarios, personalización de resultados y ordenamiento visual para mejorar la interacción con el sistema.
- Implementación de un módulo de evaluación para medir el comportamiento del recuperador mediante métricas de recuperación de información.

2 Documentación del Modelo de Recuperación Implementado

2.1 Tipo de Modelo

Se implementó un modelo de recuperación neuronal no básico basado en embeddings con Sentence Transformers. Modelo usado: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. Este modelo produce un vector denso para cada documento y para cada consulta. La recuperación se hace comparando vectores en el espacio latente. En términos de IR neuronal, este enfoque corresponde a un esquema bi-encoder (dual encoder):

- Un encoder transforma documentos a vectores.
- El mismo encoder transforma la consulta a un vector.
- La relevancia se estima con una función de similitud vectorial.

Este diseño permite indexar el corpus una vez y reutilizar los embeddings para múltiples consultas.

2.2 Modelo Híbrido: Motivación y Arquitectura

Si bien el enfoque puramente neuronal es efectivo, se ha evolucionado hacia un modelo híbrido que combina embeddings densos con indexación léxico-estadística (TF-IDF) para obtener mayor precisión y cobertura.

¿Por qué híbrido?:

1. Recuperación densa (embeddings neurales):
 - Captura relaciones semánticas profundas entre consulta y documentos.
 - Funciona bien con parafraseos y sinónimos.
2. Recuperación léxico-estadística (TF-IDF / índice invertido)
 - Excelente para coincidencias exactas de términos y consultas que dependen de palabras clave.
 - Soporta búsquedas por metadatos (directores, actores, país, temporadas).

Ventajas de la combinación

- Mayor cobertura: Algunos documentos relevantes se recuperan por semántica, otros por exactitud lexical.
- Mayor precisión: En consultas que mencionan tipos explícitamente ("dame una serie policial"), se aplican restricciones de metadatos.
- Robustez: Si la semántica falla en un ámbito, la lexical compensa (y viceversa).
- Escalabilidad: El índice léxico es ligero y rápido, permitiendo filtrado eficiente antes del ranking neural.

Arquitectura de dos etapas

- Etapa 1: Recuperación de candidatos (híbrida)
 - Búsqueda densa: se calculan similitudes sobre embeddings y se obtienen top-N candidatos (N es configurable; en código suele usarse valores como 50 por defecto).
 - Búsqueda léxica: se calcula puntuación léxica (TF-IDF / coincidencia de términos) usando el índice en `index/index.json`.
 - Fusión adaptativa: los scores dense y léxico se combinan con pesos configurables (α) y se aplican boosts/penalizaciones por metadatos (tipo, año, país, actores).
 - Filtrado por tipo: el sistema puede filtrar o penalizar resultados según la intención ("serie" vs "película").
- Etapa 2: Re-ranking neuronal (CrossEncoder)
 - Los candidatos resultantes se reevalúan con un CrossEncoder (re-ranker) que evalúa pares (consulta, documento) para producir un score fino.
 - La fusión final combina score base y score de re-ranking mediante ponderación para ordenar la lista definitiva.

2.3 Flujo Implementado en el Proyecto

1. Recolección y Limpieza: Se procesan los documentos a través del crawler/scrapper modificado, generando el archivo unificado en data/dataset.json.
2. Preprocesamiento: Construcción de una cadena semántica por documento unificando: título + géneros + directores + actores + plot
3. Indexación Dual:
 - Densa: Codificación neuronal del corpus con SentenceTransformers y almacenamiento de vectores en bd/movies_vectors.index.
 - Léxica: Procesamiento tradicional (limpieza, tokenización, stemming) y persistencia en index/index.json.
4. Flujo de ejecución RAG en tiempo de consulta:
 - El pipeline es orquestado por rag_module/pipeline.py.

- La consulta del usuario entra al retriever_wrapper.py, el cual consulta los índices y extrae el top-K de documentos candidatos empleando internamente NeuralRetriever.search_with_web_expansion()
- Si el umbral de confianza local es menor a un factor predefinido (ej. 0.72) o la cobertura léxica es baja, se dispara automáticamente el web_expander.py para recabar información en la red, integrando lo scrapeado temporalmente.
- Se transmite la información recuperada y la consulta a generator.py, quien ensambla un *prompt* y lo inyecta a un Modelo de Lenguaje Local (Ollama - Neural-Chat).
- El modelo sintetiza los datos, formulando la respuesta conversacional definitiva que lee el usuario final.

Implementación Principal y Ejecución:

La orquestación actual centralizó las operaciones en main.py, permitiendo ejecutar el recabado y la interfaz final a través de comandos escalonados:

- Crawler y Scraper: Se pueden invocar con python main.py crawl y python main.py scrape (que impactan sobre crawler/crawler.py y scraper/scraper.py).
- Recuperador Híbrido (Base): Configurado principalmente en neural_based_model/neural_retriever.py, es el motor base. Para validaciones, puede verificarse aislando el backend con python run_retrieval_demo.py.
- Evaluación RAG Asistida: Lanzada mediante python -m rag_module.run_rag, abre consola para testeos con Ollama de manera interactiva.
- Ejecución completa: Invocando python main.py.
- Interfaz visual: app.py implementa la interfaz en Streamlit.

2.4 Función de Ranking

La similitud utilizada es coseno:

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} \quad (1)$$

Donde:

- $\vec{q}$ es el embedding de la consulta.
- $\vec{d}$ es el embedding del documento.

Módulo de posicionamiento: `positioning_module/ranking.py` es el componente encargado de ajustar el orden final de los resultados, combinando señales de similitud, metadatos y criterios de relevancia visual.

Señal base: la similitud coseno entre embeddings de consulta y documento es una de las señales principales (embeddings calculados con SentenceTransformer).

Procedimiento real implementado:

1. Recuperación inicial por embeddings y por puntuación léxica.
2. Combinación preliminar de scores (neural + léxico) con pesos configurables.
3. Re-ranking con CrossEncoder para refinar orden.
4. Ajustes finales por metadatos (año, país, tipo, actores) aplicados como boosts/penalizaciones.

2.5 Análisis de Complejidad y Comportamiento

Si N es el número de documentos y D la dimensión del embedding:

- Costo de indexación inicial: $O(N \cdot C_{enc})$ donde C_{enc} es el costo del encoder por documento.
- Costo por consulta (similitud coseno vectorizada): aproximadamente $O(N \cdot D)$.
- Costo de ordenar resultados completos: $O(N \log N)$.
- Costo adicional de re-ranking: depende del número de candidatos procesados por el CrossEncoder.
- Costo adicional de expansión web: variable, condicionado por la disponibilidad de la red y por la cantidad de documentos externos recuperados.

En el corpus actual, el cálculo de similitud y ordenamiento se ejecuta con rapidez en CPU. No obstante, el tiempo total de respuesta puede aumentar por la inferencia del modelo generativo local y por la latencia asociada a la expansión web, en caso de activarse.

2.6 Criterios de Desempate y Estabilidad del Ranking

El criterio principal de ordenamiento es la puntuación final obtenida tras combinar señales semánticas, léxicas y de re-ranking. El sistema busca mantener un comportamiento estable en la presentación de resultados, de modo que pequeñas variaciones numéricas no provoquen cambios drásticos en el orden.

En la implementación final (Corte 3), el desempate se apoya en la ordenación interna de los resultados y en la combinación de señales secundarias derivadas de metadatos. Entre estas señales se incluyen coincidencias con actores, géneros, tipo de contenido, país, año y otros atributos relevantes.

El sistema prioriza los siguientes criterios de forma general:

1. Relevancia final combinada.

2. Relevancia semántica base.
3. Coincidencia léxica.
4. Señales de metadatos.
5. Tipo de contenido solicitado.
6. Identificador estable del documento como criterio final de consistencia.

Este enfoque favorece la estabilidad del ranking y mejora la experiencia de usuario al evitar reordenamientos arbitrarios entre consultas similares.

3 Diseño de la interfaz visual y posicionamiento de la información

La interfaz visual del sistema fue diseñada con el objetivo de facilitar la exploración del corpus de forma clara, jerarquizada e intuitiva. Para ello, se optó por una presentación en tarjetas, donde cada resultado se muestra como una unidad visual independiente que resume los atributos más relevantes del documento: título, tipo de contenido, puntuación, año, valoración y sinopsis.

En términos de distribución visual, la información se organiza siguiendo una jerarquía de atención. En primer lugar, se ubican los elementos asociados a la relevancia de la búsqueda, como la posición en el ranking y la puntuación obtenida. Posteriormente, se presentan los metadatos descriptivos del contenido, lo que permite identificar con rapidez si el documento recuperado responde a la intención de la consulta. Finalmente, la sinopsis aparece como un elemento complementario para profundizar en la comprensión del resultado.

Asimismo, el panel lateral concentra los controles de búsqueda, los parámetros de recuperación y los ejemplos de consultas, con el propósito de separar claramente las acciones de configuración respecto de la visualización de resultados. Esta organización mejora la legibilidad general de la interfaz y favorece una interacción más ordenada con el sistema.

4 Justificación de las estrategias de posicionamiento implementadas

La estrategia de posicionamiento adoptada en el proyecto busca un equilibrio práctico entre precisión, cobertura y experiencia de usuario. Se fundamenta en cuatro decisiones principales, cada una respaldada por componentes concretos del código:

1. Hibridación semántica-léxica: la fusión de señales neurales y léxicas se implementa en `compute_hybrid_fusion` (`positioning_module/ranking.py`) y se controla por el parámetro `Alpha`. Esta decisión garantiza que el sistema capture intención (embeddings) y, al mismo tiempo, mantenga precisión en búsquedas por entidades o términos exactos (índice léxico).
2. Re-ranking fino con Cross-Encoder: cuando la topología de candidatos presenta ambigüedad, aplicamos un re-ranking por Cross-Encoder y lo mezclamos con el score base mediante `compute_rerank_fusion` y

NeuralRetriever.rerank_candidates. Así se resuelven empates semánticos y se obtiene mayor coherencia en el top-K final.

3. Prioridad de metadata y frescura: el posicionamiento aplica boosts y penalizaciones explícitas para título, director/creador, actores, temporadas y año (funciones `metadata_match_score`, `actor_alignment_score`, `season_alignment_score`, `year_alignment_score` en `positioning_module/ranking.py`). Además, `_freshness_score` incorpora una señal temporal que favorece contenido más reciente cuando procede.
4. Robustez y cobertura externa: el sistema monitoriza la cobertura léxica local (`query_lexical_coverage`) y, si la confianza es baja, activa `search_with_web_expansion` para traer documentos web adicionales (`web_search/`).

Complementariamente, la personalización (módulo `recommendation_module`) respeta la señal del retriever como prioritaria y re-puntúa resultados usando historial, géneros y vistas, preservando top-N en búsquedas de metadata específica. Para desempates y trazabilidad, el tie-breaker determinista (`build_positioning_tie_breaker`) combina completitud de metadata, match de género y posición original, lo que produce un orden reproducible y explicable.

5 Módulos adicionales de recomendación y evaluación

Además del motor principal de recuperación, el sistema incorpora un módulo de recomendación orientado a personalizar la experiencia del usuario. Este componente utiliza el historial de búsquedas, las interacciones previas y las preferencias registradas para ajustar la selección de resultados y ofrecer sugerencias más cercanas a los intereses de cada persona. De esta manera, el sistema no solo recupera contenido relevante para la consulta actual, sino que también adapta parte de la salida al perfil del usuario.

El proyecto también incluye un módulo de evaluación que permite analizar el comportamiento del recuperador mediante métricas propias de recuperación de información. Gracias a este componente es posible medir la calidad de los resultados, comparar el desempeño entre distintas consultas y verificar si las estrategias de búsqueda y ordenamiento mantienen un nivel adecuado de relevancia. En conjunto, esta evaluación aporta una base objetiva para revisar el funcionamiento del sistema y justificar futuras mejoras.

6 Estructura de la implementación

El sistema CulturaSearch está organizado en módulos independientes pero integrados, cada uno con una responsabilidad específica dentro del flujo general de recuperación de información.

- `crawler/`: descubre Urls de películas y series a partir de fuentes semilla.
- `scraper/`: extrae y estructura los metadatos de las páginas descubiertas.

- `index/`: construye y limpia el índice léxico invertido.
- `bd/`: almacena el índice vectorial, embeddings y metadatos asociados.
- `neural_based_model/`: implementa el recuperador neuronal y el re-ranking.
- `positioning_module/`: gestiona puntuaciones combinadas y estrategias de posicionamiento.
- `rag_module/`: coordina la recuperación aumentada con generación de texto.
- `web_search/`: permite expansión web cuando la confianza local es baja.
- `recommendation_module/`: gestiona el perfil del usuario y la personalización.
- `evaluation_module/`: mide el desempeño del sistema con métricas de recuperación.
- `estadisticas/`: contiene scripts para análisis cuantitativo del corpus.
- `app.py`: implementa la interfaz visual del sistema.
- `main.py`: actúa como lanzador general para los componentes principales.

Esta organización modular permite mantener una arquitectura escalable, reutilizable y fácil de depurar.

7 Opinión crítica del proyecto implementado

Desde una perspectiva crítica, el proyecto presenta una evolución sólida y coherente respecto a una recuperación básica de información. Entre sus principales bondades se destaca la integración de múltiples enfoques complementarios: recuperación semántica con embeddings, señales léxicas, re-ranking, expansión web, interfaz visual y personalización basada en el perfil del usuario. Esta combinación convierte al sistema en una plataforma más completa y cercana a escenarios reales de búsqueda y exploración de contenido.

No obstante, también se identifican insuficiencias. La primera de ellas es la dependencia de componentes externos, como Ollama y la conectividad web, lo cual puede afectar la disponibilidad y la latencia del sistema. En segundo lugar, la complejidad de la arquitectura incrementa el costo de mantenimiento y hace necesario documentar con precisión cada módulo para facilitar su evolución futura. Finalmente, aunque la interfaz mejora notablemente la experiencia de uso, su efectividad depende en gran medida de la calidad de los datos recuperados y de la consistencia de los metadatos disponibles en el corpus.

8 Deficiencias detectadas por el equipo y posibles soluciones

- Dependencia del modelo local: Si Ollama falla, el sistema deja de responder. Se propone añadir mecanismos de respaldo para que la funcionalidad se degrade sin interrumpirse por completo.
- Riesgos de la expansión web: La búsqueda en internet depende de la red y de fuentes externas. Se sugiere mejorar el uso de caché, reforzar la detección de errores y limitar cuándo se activa la búsqueda online.

- Personalización limitada al inicio: El sistema personaliza mejor tras varias interacciones; al principio el perfil es pobre. Una solución sería incluir una configuración inicial o preferencias básicas para construir un perfil mínimo.
- Mantenimiento del corpus y documentación: Los índices y la documentación deben actualizarse junto con el corpus. Se recomienda automatizar la reconstrucción de índices y revisar periódicamente la documentación técnica.

9 Estadísticas Básicas del Corpus Recopilado

9.1 Tamaño General del Corpus

Tras el proceso de crawling y scraping realizado, se ha consolidado un corpus con las siguientes métricas de volumen:

- Cantidad total de documentos: 2684.
- Documentos tipo película: 1541.
- Documentos tipo serie: 1143.

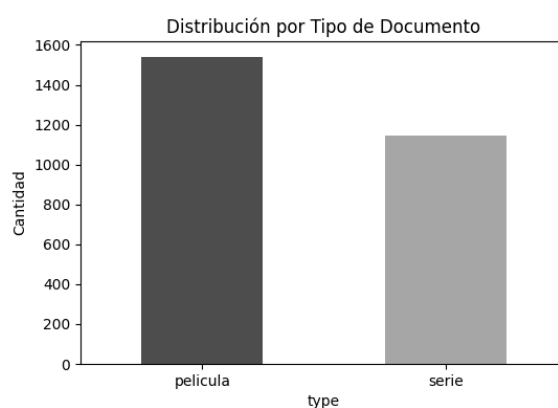


Fig. 1. Distribución comparativa entre películas y series de TV recopiladas en el dominio de cultura y entretenimiento.

9.2 Análisis de Longitud Textual

Para garantizar que el modelo de recuperación semántica (embeddings) reciba suficiente contexto, se analizó la extensión de la cadena de texto consolidada título + sinopsis. Los resultados actualizados del corpus son:

- Promedio de palabras por documento: 138.90
- Promedio de caracteres por documento: 829.92
- Mínimo de palabras en un documento: 1
- Máximo de palabras en un documento: 520

9.3 Diversidad y Riqueza del Corpus

El dominio de cultura y entretenimiento presenta una alta variabilidad en sus atributos. El análisis de los metadatos extraídos de las fuentes especializadas arroja los siguientes indicadores de diversidad, fundamentales para validar la robustez del sistema de recuperación:

Distribución Temática y Geográfica

- Géneros detectados: Se identificaron 34 categorías únicas. El corpus presenta una marcada predominancia de los géneros Drama (1102 documentos) y Acción (666), seguidos por Comedia (598), Suspense (495) y Aventura (441).
- Alcance Global: El sistema integra obras de 41 países de origen. Destaca la hegemonía de producciones de Estados Unidos (582), seguida por una presencia significativa de España (128) y Gran Bretaña (86).

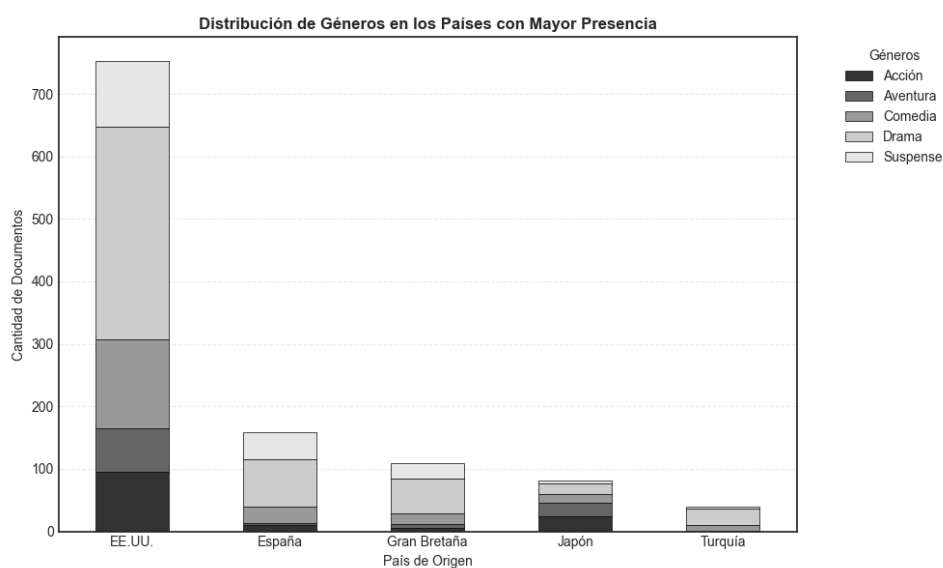


Fig. 2. Distribución de los cinco géneros principales segmentados por país de origen. La visualización permite confirmar la diversidad del corpus y la representatividad de cada categoría dentro de los mercados cinematográficos líderes.

Entidades y Personalidades

- Directores: Se identificaron 1190 directores únicos
- Elenco: El corpus cuenta con una base de 4761 actores únicos.

Valoración y Calidad (Ratings)

- Rating promedio: La calificación media de las obras en el corpus es de 3.55.
- Rango de valoración: El conjunto de datos abarca desde obras de alto impacto crítico (1.2) hasta producciones de menor recepción (4.7).

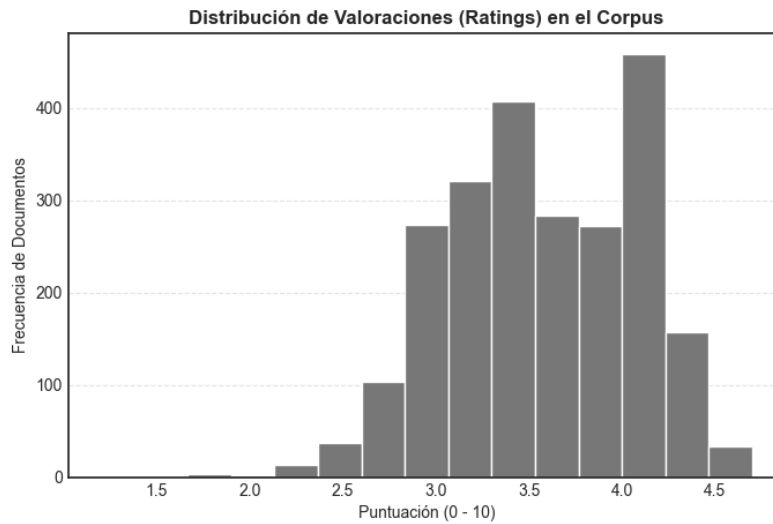


Fig. 3. Distribución de frecuencias de las calificaciones en el corpus.

9.4 Análisis del Rango Temporal

El corpus recolectado presenta una ventana cronológica que permite evaluar el desempeño del sistema ante diferentes contextos de producción audiovisual. Los datos temporales registrados son:

- Año mínimo detectado: 1944
- Año máximo detectado: 2028
- Cobertura temporal: El conjunto de datos abarca un período de 84 años.

10 Fuentes Bibliográficas

1. Mitra, B., y Craswell, N. (2018). An Introduction to Neural Information Retrieval. Foundations and Trends in Information Retrieval
2. NLTK Project. NLTK Documentation: Spanish Snowball Stemmer. Recuperado de: <https://www.nltk.org/>
3. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)
4. Hugging Face Documentation: paraphrase-multilingual-MiniLM-L12-v2. Available at: huggingface.co.